

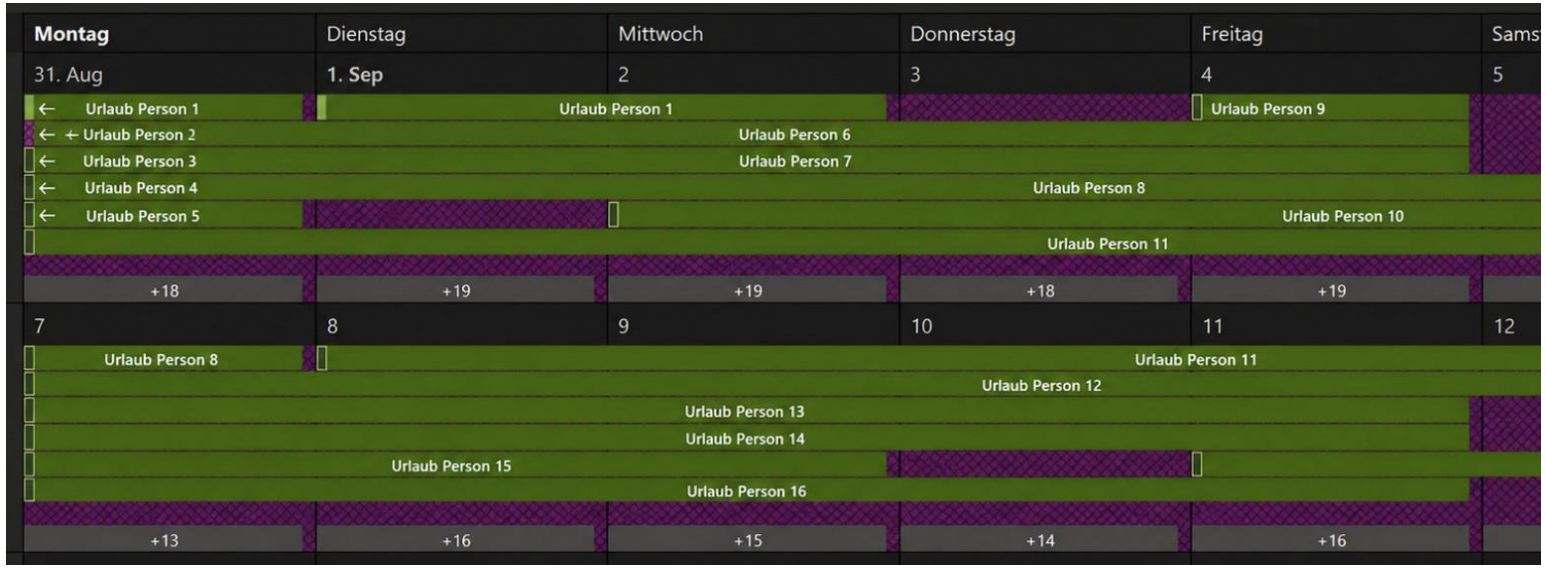
ProVaCaSy
HOCHSCHULE ESSLINGEN

Osman Sallamaci, Tommy Luu, Dominique Sluga

ProVaCaSy – Professoren Vacation Calender System

Aktueller Stand: Outlook und Handarbeit

ProVaCaSy



Dezentrale Pflege

Isolierte Kalendereinträge

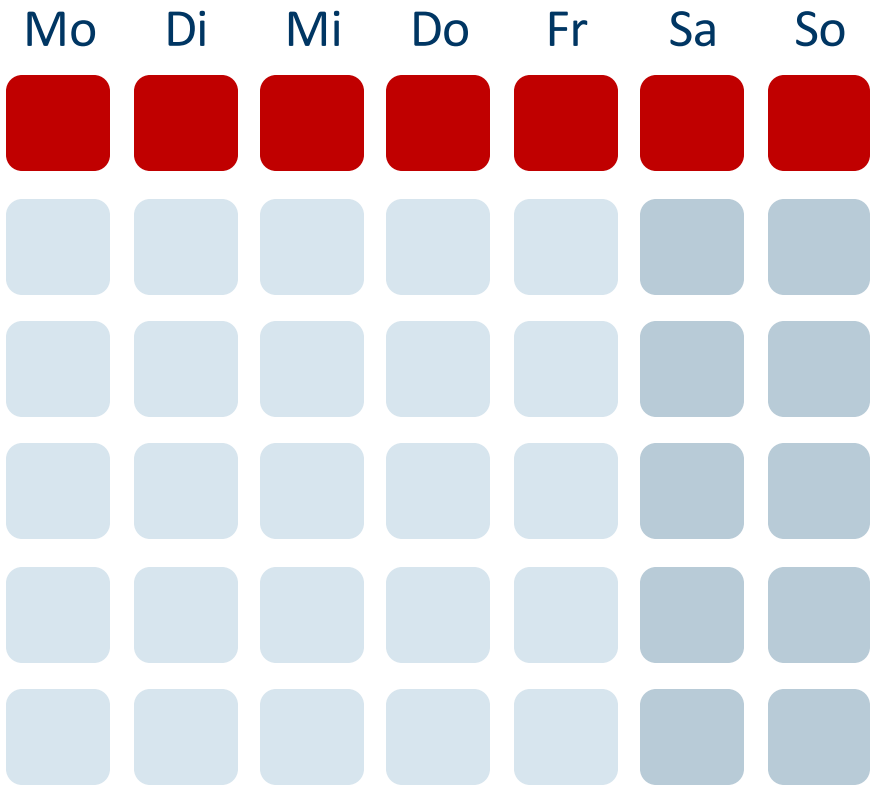
Fehleranfälligkeit

Gefahr von Doppelbuchungen

Fehlende Übersicht

Resturlaub nicht einsehbar

Agenda



I. Anforderungen an die Anwendung

II. High-Level-Architektur

III. Technologien & Tech-Stack

IV. Entwicklungsumgebung & Toolchain

V. Nutzerinteraktion & UI Mockups

VI. Teamzusammenarbeit: Methode, Rollen & Tools

Ziel: Der Soll Zustand

ProVaCaSy

- **Zentrale Webplattform**
- **Automatische Berechnung**
- **Integrierte Validierung**
- **Hochschul-Login**

30

Urlaubstage pro Jahr

Automatisch berechnet mit Feiertagen für Baden-Württemberg

Zielgruppen und Benutzerrollen

ProVaCaSy

ProVaCaSy wurde speziell für Professoren, das Sekretariat und das Dekanat der Hochschule Esslingen entwickelt und unterstützt drei unterschiedliche Benutzerrollen



Professor (Self-Service)

Trägt Urlaube und Konferenzen ein, sieht Resturlaub, sowie den Teamkalender



Sekretariat (Einsicht)

Kann Abwesenheiten stellvertretend eintragen und Statistiken einsehen



Dekanat (Freigabe)

Kann Abwesenheiten stellvertretend eintragen, Statistiken einsehen und Abwesenheiten genehmigen oder ablehnen

Die wichtigsten Must-Haves (MVP)

ProVaCaSy

Keycloak SSO & RBAC
Sicherer Login mit automatischer Rollenerkennung

Abwesenheitsverwaltung
Erfassung von Urlaub / Konferenz inklusive automatischer Doppelbuchungsprüfung, sowie optische Warnung ab 30 Tagen

ICS-Integration
Bereitstellung von ICS-Dateien zur unkomplizierten Übernahme von Terminen in den Outlook-Kalender

Kontingent & Feiertage
Dashboard des persönlichen Urlaubskontigents, dynamische Feiertagsberechnung für Baden-Württemberg

Nice-to-Have Features

ProVaCaSy



Direkte Outlook-Extension

Plattformübergreifende Outlook-Erweiterung zur komfortablen Verwaltung und direkte Synchronisation



Erweiterte Statistik-Exporte

Grafische Auswertungen sowie direkter Datenexport als CSV- oder PDF-Bericht

Nicht Funktionale Anforderungen

ProVaCaSy

Datenschutz & Lokalität

Keine externen Ressourcen im Frontend; sämtliche Datenabrufe laufen über das Backend

Wartbarkeit & Clean Code

Automatisierte Code-Analyse durch SonarQube sowie Unit- und Integrationstests

Usability & Design

Light-Mode / Dark-Mode-Toggle mit unkomplizierter Bedienung

Portabilität & Betrieb

Vollständige Containerisierung mit Docker Compose für standardisierten Betrieb

Agenda

I. Anforderungen an die Anwendung

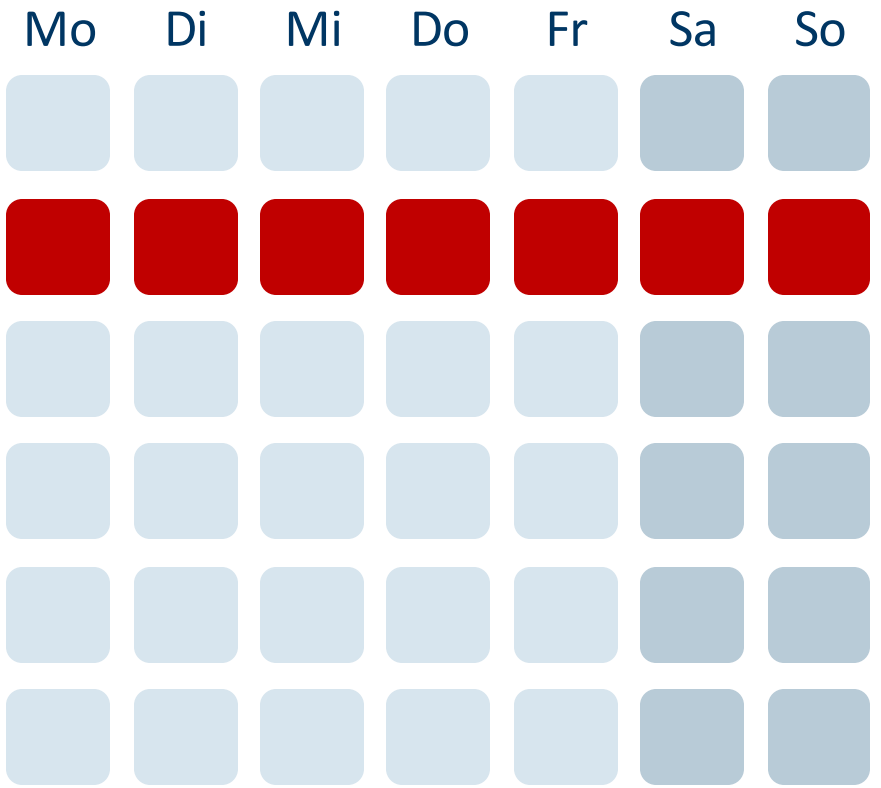
II. High-Level-Architektur

III. Technologien & Tech-Stack

IV. Entwicklungsumgebung & Toolchain

V. Nutzerinteraktion & UI Mockups

VI. Teamzusammenarbeit: Methode, Rollen & Tools



Architekturstil

ProVaCaSy

Die 3-Schichten-Architektur ermöglicht eine klare Aufgabentrennung.

01 PRESENTATION LAYER
Frontend
Kalenderansichten, Dashboards und Eingabemasken

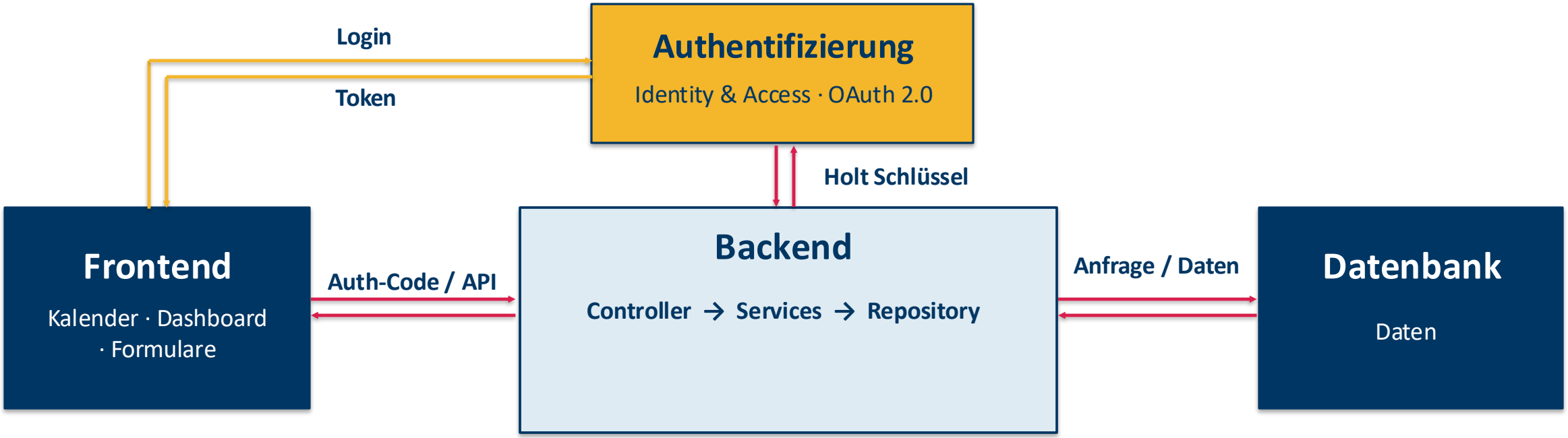
02 BUSINESS LOGIC LAYER
Backend
Urlaubslogik, Kontingente, Rollen und Berechtigungen

03 DATA ACCESS LAYER
Datenbank
Dauerhafte, konsistente Speicherung von Benutzerdaten, Urlaubsanträgen, Abwesenheiten und Systemkonfigurationen

Kommunikation

ProVaCaSy

Klare Schnittstellen verbinden Benutzeroberfläche, Fachlogik, Datenhaltung und Anmeldung



Sicherheit

ProVaCaSy

Sichere Anmeldung

Zentraler Login für Professoren, Sekretariat und Dekanat

Rollenbasierter Zugriff

Jede Rolle sieht nur die vorgesehenen Funktionen und Daten.

Gültige Tokens

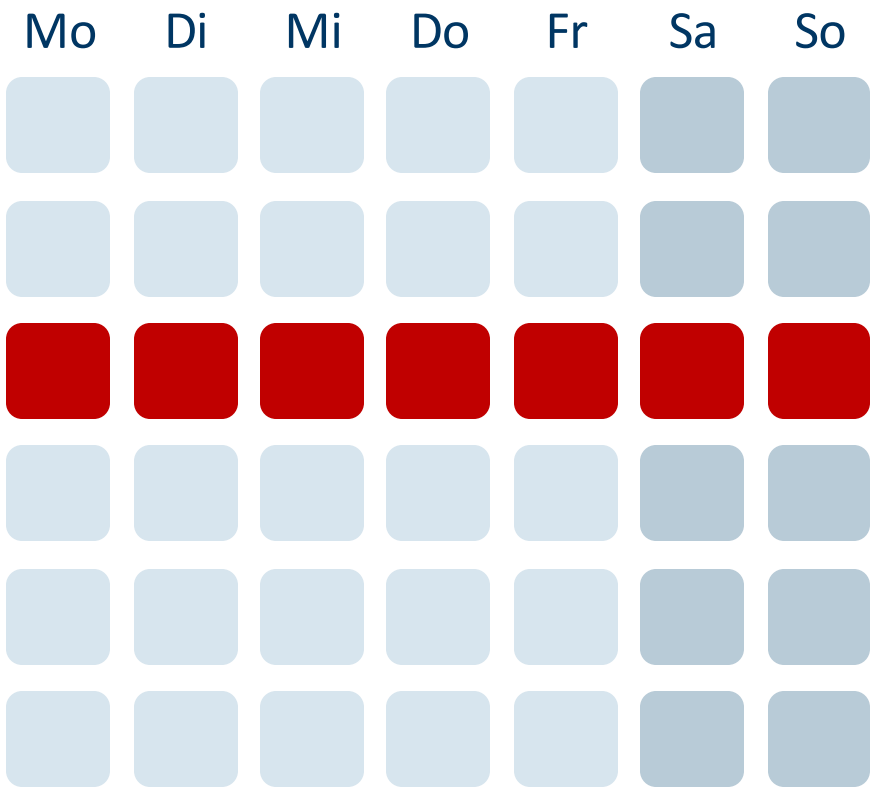
Das Backend lehnt Anfragen ohne gültiges Token ab.

Berechtigungen im Projekt

Professoren verwalten eigene Anträge; nur das Dekanat darf Anträge genehmigen oder ablehnen.

Agenda

I. Anforderungen an die Anwendung
II. High-Level-Architektur
III. Technologien & Tech-Stack
IV. Entwicklungsumgebung & Toolchain
V. Nutzerinteraktion & UI Mockups
VI. Teamzusammenarbeit: Methode, Rollen & Tools



Frontend



- **Single Page Application** — ohne Neuladen
- **Komponenten** — wiederverwendbar
- **Einstieg** — schnell im Team
- **Architektur** — leichtgewichtig & klar
- **Ökosystem** — große Community
- **Pakete** — lokal über npm

Backend



- **Schichten** — klar getrennt mit DTOs
- **API** — REST & Validierung
- **Sicherheit** — Keycloak / OAuth2
- **Tests** — Unit & Integration
- **Deployment** — eingebetteter Webserver
- **Setup** — schnell, wenig Konfiguration
- **Know-how** — bekanntes Ökosystem & gute Doku

Datenbank

- **Datenmodell** — relational
- **Transaktionen** — zuverlässig
- **Anbindung** — über ORM
- **Betrieb** — Docker-Container
- **Zeitlogik** — Datum, Urlaub & Überschneidungen





Authentifizierung

- **Sicherheit** — bewährtes Produkt
- **Login** — zentral statt Eigenentwicklung
- **Standards** — OAuth2 & OpenID Connect
- **Integration** — Spring Boot & Vue.js
- **Rollen** — zentral verwaltet
- **Zugang** — Single Sign-On
- **Betrieb** — lokal
- **Datenschutz** — kein externer Dienst



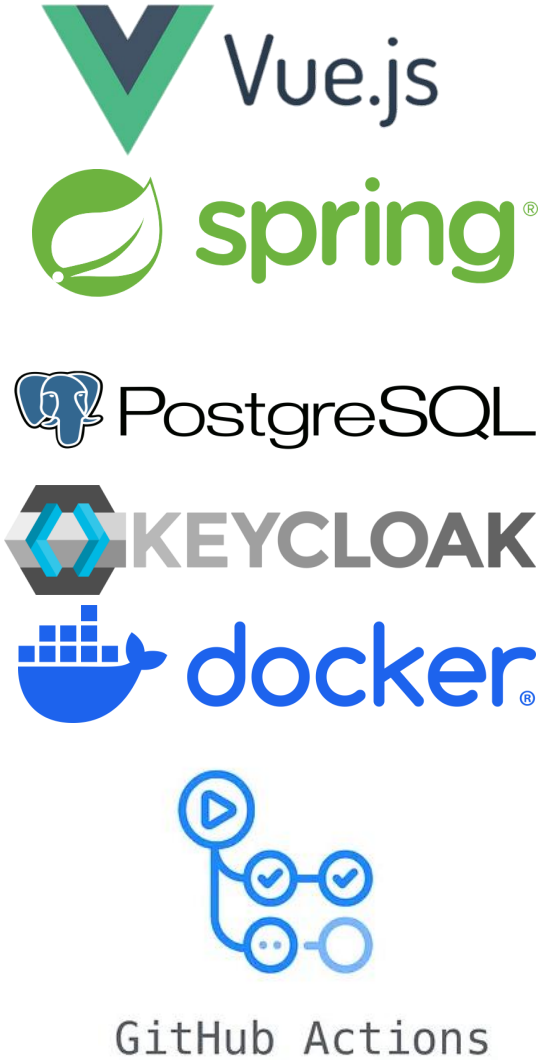
Betrieb

- **Umgebung** — auf jedem Rechner identisch
- **Dienste** — je ein Container
- **Start** — gesamtes System mit einem Befehl
- **Onboarding** — schneller Teameinstieg
- **Architektur** — isolierte Services
- **Updates** — neue Images via GitHub Packages



Pipeline

- **Build & Tests** — automatisiert
- **Images** — Docker via GitHub Packages
- **Effizienz** — nur geänderte Dienste neu bauen
- **Integration** — direkt in GitHub
- **Merge** — erst nach erfolgreicher Pipeline
- **Team** — einheitlicher Ablauf
- **Fokus** — weniger Handarbeit



Qualität

- **Pipeline** — automatische Codeprüfung
- **Fehler** — früh erkennen
- **Wartbarkeit** — Code Smells & Lesbarkeit
- **Sicherheit** — Lücken erkennen
- **Duplikate** — doppelten Code finden











GitHub Actions



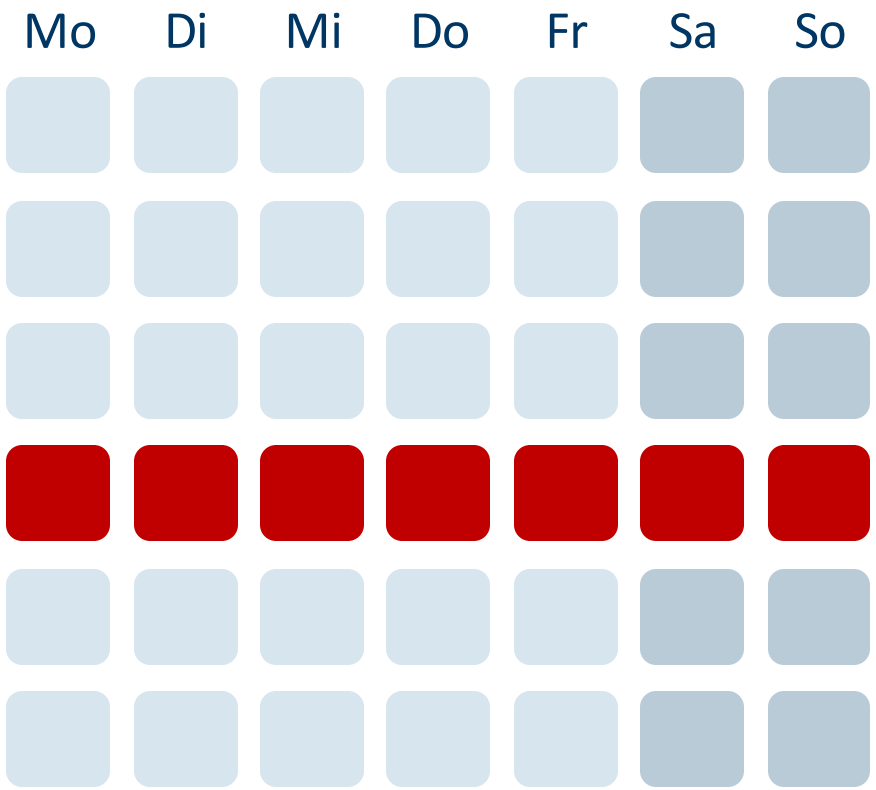
Zusammenfassung

- **Frontend** — Vue.js
- **Backend** — Spring Boot
- **Datenbank** — PostgreSQL
- **Authentifizierung** — Keycloak
- **Betrieb** — Docker
- **Pipeline** — GitHub Actions
- **Codequalität** — SonarQube



Agenda

I. Anforderungen an die Anwendung
II. High-Level-Architektur
III. Technologien & Tech-Stack
IV. Entwicklungsumgebung & Toolchain
V. Nutzerinteraktion & UI Mockups
VI. Teamzusammenarbeit: Methode, Rollen & Tools



Entwicklungsumgebung

ProVaCaSy



- Einheitliche IDE
- Git und Docker integriert

Versionsmanagement: GitHub

ProVaCaSy

- Getrennte Repositories für Frontend, Backend und Doku
- Feature-Branches, geschützter Main
- Vier-Augen-Prinzip





API-first: OpenAPI-Spezifikation

ProVaCaSy

- Schnittstellenvertrag zuerst
- Automatisierte Generierung
- Unabhängige Entwicklung







Übersicht

ProVaCaSy



Frontend & Backend IDE:
Visual Studio Code



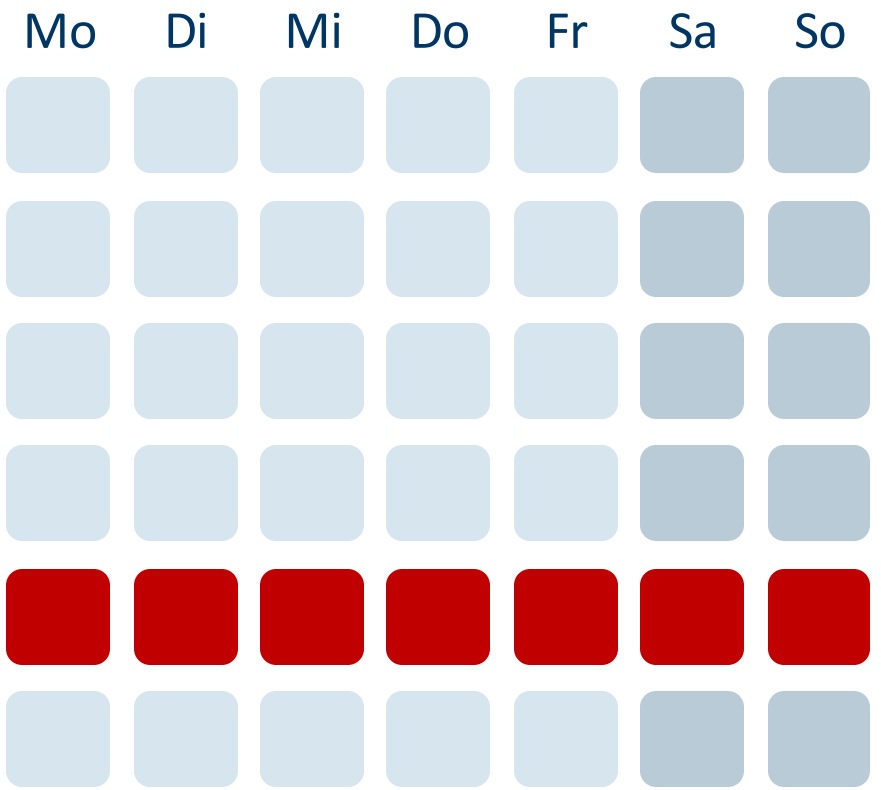
Versionsmanagement:
GitHub



API-Spezifikation:
OpenAPI

Agenda

I. Anforderungen an die Anwendung
II. High-Level-Architektur
III. Technologien & Tech-Stack
IV. Entwicklungsumgebung & Toolchain
V. Nutzerinteraktion & UI Mockups
VI. Teamzusammenarbeit: Methode, Rollen & Tools



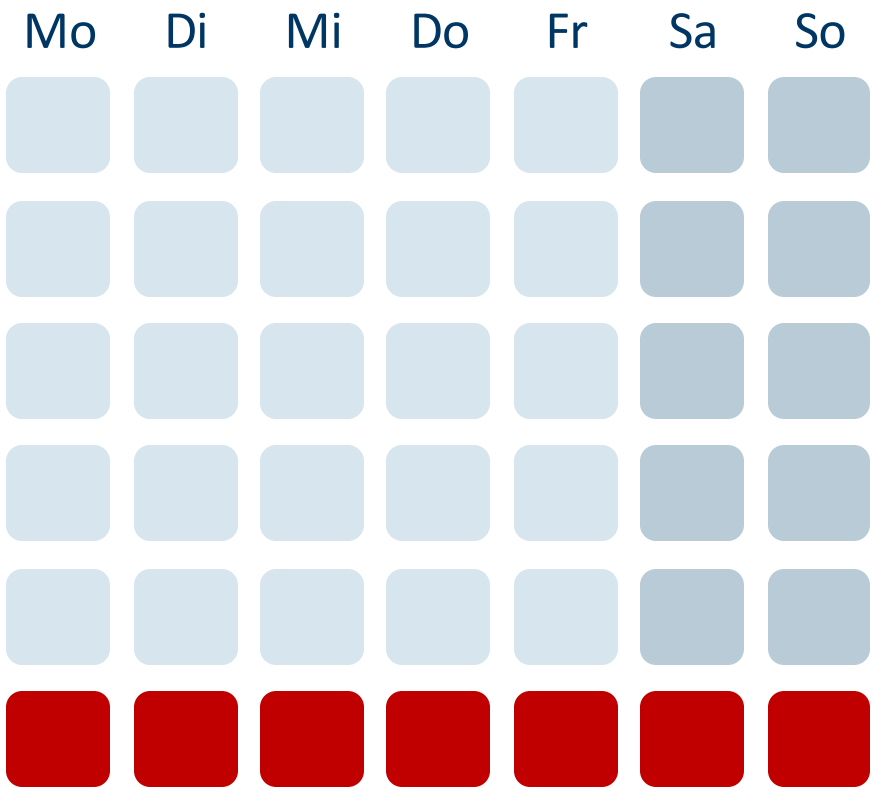
Mockup-Demo

ProVaCaSy

Mockup-Demo starten

Agenda

I. Anforderungen an die Anwendung
II. High-Level-Architektur
III. Technologien & Tech-Stack
IV. Entwicklungsumgebung & Toolchain
V. Nutzerinteraktion & UI Mockups
VI. Teamzusammenarbeit: Methode, Rollen & Tools



Methode

ProVaCaSy

Kanban schafft Transparenz und hält die Arbeit im Team fokussiert.

01

GitHub Issues

Aufgaben zentral erfassen

02

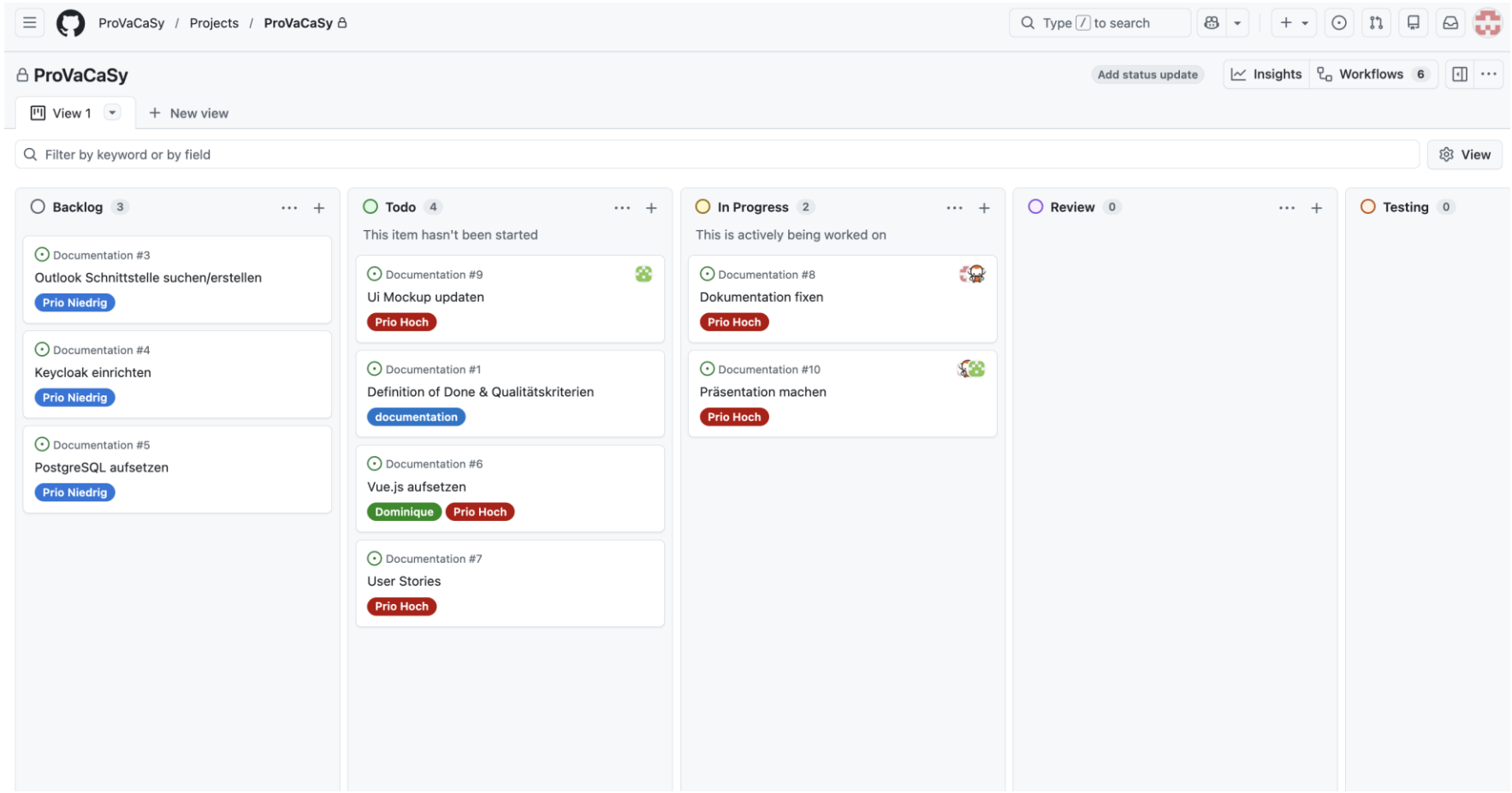
Klare Schritte

Backlog → Todo → In Progress → Review → Testing → Done

03

Gemeinsamer Fokus

Viele Aufgaben parallel; Blockaden früh klären



Definition of Done

ProVaCaSy

Eine Aufgabe ist erst abgeschlossen, wenn alle fünf Kriterien erfüllt sind.

01	**Entwicklung**	Anforderungen und Akzeptanzkriterien umgesetzt; Clean Code und Schichtentrennung eingehalten.
02	**Tests**	Unit- und Integrationstests vorhanden; Pipeline fehlerfrei; Frontend mit Testskriptbuch geprüft.
03	**Dokumentation**	API-Beschreibung und LaTeX-Dokumentation aktualisiert.
04	**Vier-Augen-Prinzip**	Ein weiteres Teammitglied hat die Änderung geprüft und freigegeben.
05	**Aufwand**	Aufgewendete Zeit ist im zugehörigen Issue eingetragen.

Aufgabenverteilung

ProVaCaSy

Jedes Teammitglied verantwortet einen klaren Bereich.

OS

Osman Sallamaci

Backend & DevOps

TL

Tommy Luu

Backend & Keycloak

DS

Dominique Sluga

Frontend

GEMEINSAM Abstimmung · Code-Reviews · Tests · Dokumentation

Kommunikations- & Dokumentations- Tools

ProVaCaSy

Klare Kanäle für schnelle Abstimmung und nachvollziehbare Dokumentation.

01

IM TEAM

Discord
WhatsApp

Tägliche Abstimmung

02

BETREUER & KUNDE

Webex
E-Mail

Termine und Rückfragen

03

DOKUMENTATION

Fachschaften.org
HajTeX

Gemeinsame Projektdokumentation

Quellen

ProVaCaSy

Vue.js Logo
https://www.stickpng.com/de/img/icons-logos-emojis/ikonische-marken/vuejs-logo-landschaft

Spring Boot Logo
https://spring.io/trademarks

PostgreSQL Logo
https://1000logos.net/postgresql-logo/

Keycloak Logo
https://seeklogo.com/vector-logo/505405/keycloak

Docker Logo
https://de.wikipedia.org/wiki/Datei:Docker_Logo.png

GitHub Actions Logo
https://blog.devops.dev/basics-of-github-actions-c73eb9f16211

Visual Studio Code Logo
https://de.wikipedia.org/wiki/Datei:Visual_Studio_Code_1.35_icon.svg#/media/Datei:Visual_Studio_Code_1.35_icon.svg

SonarQube Logo
https://www.testautomatisierung.org/lexikon/sonarqube/sonarqube-logo-black-128-px/

GitHub Logo
https://en.wikipedia.org/wiki/GitHub#/media/File:GitHub_Invertocat_Logo.svg

Bilder Nutzerrollen
KI generiert von Gemini

OpenApi Logo
https://thesvg.org/icon/openapi

Vielen Dank für Ihre Aufmerksamkeit

Noch Fragen?

Buildprozess: vom Push zum Docker-Image

ProVaCaSy

Nur geprüfter und freigegebener Code erreicht main und die Registry.



FEATURE-BRANCH

Push & Pull Request

Branch zu GitHub pushen. PR nach main öffnen und Issue verknüpfen.

AUTOMATISCH

GitHub Actions

Der Pull Request startet die Pipeline ohne manuellen Eingriff.

QUALITY GATE

Build, Tests & SonarQube

Projekt bauen, Tests ausführen und Codequalität mit SonarQube prüfen.

VIER-AUGEN-PRINZIP

Review & Merge

Zweite Person prüft. Merge nur bei grüner Pipeline und Freigabe.

NACH DEM MERGE

Docker veröffentlichen

Images erstellen und versioniert in GitHub Packages ablegen.

ERGEBNIS Geschützter main-Branch · geprüfter Code · versionierte Docker-Images in GitHub Packages

Nächste Schritte

ProVaCaSy

Bis zum Meilenstein 3 am 08.12.2026 setzen wir fünf Schritte um.

01	**Konzept**	Meilenstein 2 am 17.11.2026: User Stories und technisches Konzept abgeben.
02	**OpenAPI**	API-Spezifikation festlegen und Clients für Frontend und Backend generieren.
03	**Keycloak**	Login mit Rollen aufsetzen; Anbindung an den Hochschul-Login mit dem Kunden klären.
04	**Pipeline**	Build, Tests und SonarQube in GitHub Actions einrichten; Docker-Images in GitHub Packages.
05	**Prototyp**	Prototyp mit Login bis 08.12.2026; Mockups mit dem Kunden abstimmen.

Rolle: Professor

ProVaCaSy

Bedürfnisse:

- Eine benutzerfreundliche und übersichtliche Benutzeroberfläche
- Zuverlässige, zeitnahe Rückmeldungen und Benachrichtigungen
- Transparente Einsicht in Genehmigungsstände



Prof. Dr. Klaus Günther Jost

Probleme:

- Hoher Zeitverlust durch umständliche oder unübersichtliche Prozesse
- Unklarheit über verbleibende Urlaubstage und Bearbeitungsstände
- Fehlen einer zentralen und einfachen Lösung für Anträge

Rolle: Sekretär

ProVaCaSy

Bedürfnisse:

- Zentrales und übersichtliches Dashboard für alle Bereichsanträge
- Schnelle und transparente Freigabeprozesse
- Zuverlässige Benachrichtigungen bei Statusänderungen



Anna Schmidt

Probleme:

- Hoher Aufwand durch manuelle und unübersichtliche Antragswege
- Lange Wartezeiten und unklare Bearbeitungsstände
- Fehlende zentrale Übersicht über das Personal

Rolle: Dekan

ProVaCaSy

Bedürfnisse:

- Zentrales Dashboard zur schnellen Prüfung und Freigabe
- Verlässliche Kennzahlen zur Personalverfügbarkeit im Fachbereich
- Transparente Historie und Nachvollziehbarkeit aller Freigaben



Dr. Paul Hoffmann

Probleme:

- Verzögerungen durch intransparente oder papierbasierte Dienstwege
- Hoher Abstimmungs- und Nachfragebedarf bei Anträgen
- Mangelnde Gesamtübersicht über die personelle Auslastung

UX-Konzept & Visual Design

ProVaCaSy

Ziel:

- Statistiken kompakt einsehen
- Prozesse vereinfachen
- Gebrauchstaugliches Design

Anforderungen an Usability:

- Benutzungsoberfläche selbsterklärend
- Funktionen sind schnell & einfach auffindbar

Kernfunktionen:

- Digitale Anträge in Webapplikation
- Schnell und einfach Abwesenheiten eintragen
- Einfache Navigation